

Project Report: Group 8

Predicting Food Groups Based On Their Nutritional Values

presented by
Ruyue Wang
Yuxin Zhou
Bilegjargal Battulga
Marian Reuss
Lukas Schönwiese

submitted to the
Data and Web Science Group
Prof. Dr. Heiko Paulheim
University of Mannheim

December 2nd, 2018

Application Area and Goals

In our project, we aim to create a model that will predict a food group based on its nutritional values. The application areas for this problem are manifold. The classifier may be of use whenever desired or observed nutritional values are known beforehand and the respective food is to be determined. For example, when animals need to be fed according to a certain diet, one would be able to identify the types of food which fit the requirements best.

Given the structure of the underlying data, we identify two major challenges. The first one emerges from the fact that many categories bear great resemblance with each other, the second one from the circumstance that the number of classes we are trying to predict is quite high and there are many features. We will discuss these problems and our approaches to them more in-depth in the later sections of this report. Nevertheless, our goal is to create a classifier that delivers sufficiently strong results on the test set.

In the remainder of this report, we will proceed as follows: first, we will discuss the data set in greater detail, explain its structure and highlight some of its particularities. Then, we will explain the necessary pre-processing steps we derived from said characteristics and how we applied them to the data set in order to prepare it for the data mining process. Subsequently, we will discuss our data mining procedures in-depth and how we approached the process of finding a sufficiently strong classifier configuration. Ultimately, we will compare the classifiers' performance against a competitive baseline and against each other and discuss our findings. In doing so, we will also return to some questions and expectations that we put forward in our initial project proposal.

The National Nutrient Data Base

For our project, we use the USDA National Nutrient Data Base (NNBD) (Kelly, 2016). The data set contains information about various types of food with one specific food representing one instance. Apart from the foods' descriptions and unique IDs, the data set also contains a series of 23 nutritional values for each instance such as protein, fat or calcium. Values are listed as intake when consuming 100 grams of the respective food. It must be noted that the nutritional columns differ with respect to the unit in which they are listed. E.g., calcium is listed in milligrams, whereas phosphorus is listed in micrograms.

Moreover, the data set contains 15 columns with recommendations for a daily dietary intake for some of the nutritional values (suffixed with "_USRDA"). These columns are defined as follows: they list what share of the recommended daily

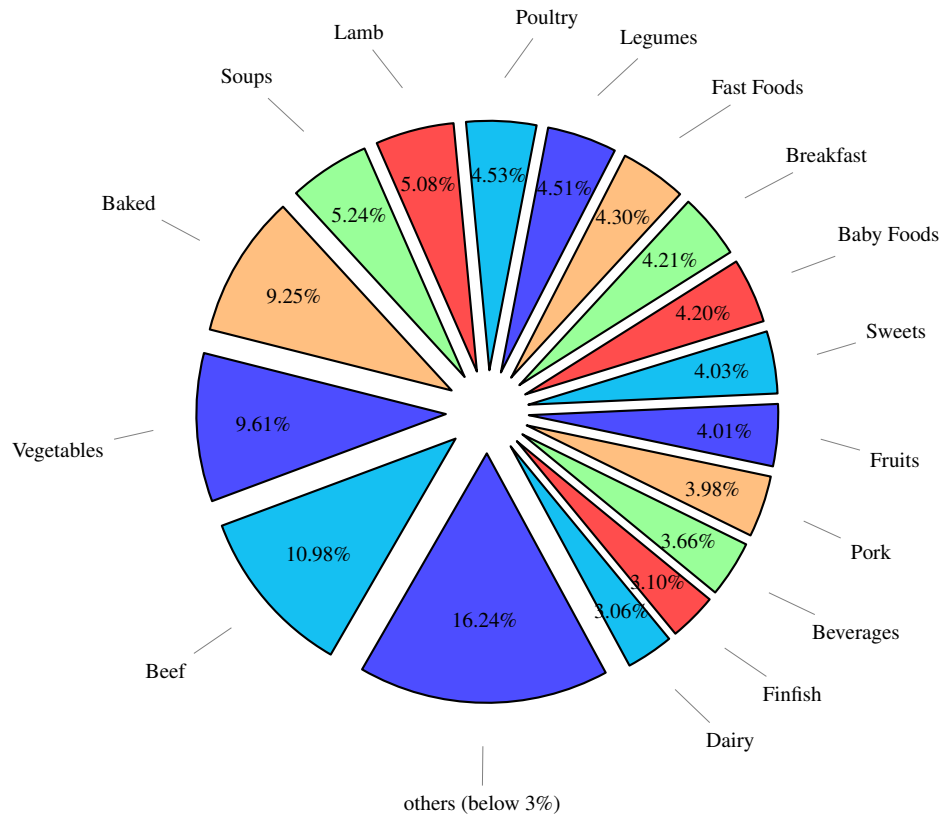


Figure 1: Class distribution prior over-sampling, abbreviated names

intake for the respective nutritional value is covered by consuming 100g of the food listed in the instance. Note, however, that recommendations for eight out of the 23 nutrients are missing.

Finally, instances are also pooled into one of 25 food groups - the label we are trying to predict. These groups range from categories where differentiation is fairly straight-forward, e.g. dairy and egg products, fats and oils, or sweets, to some that could be considered as a supercategory of others. For example, Restaurant Foods which may contain a variety of different foods. In total, there are 8618 instances (rows) and 45 columns. Therefore, assuming we used everything but the food group label and auxiliary information (Description, ShortDescription, ID and so forth) as features for a classifier, this would sum up to a total of 38 features as input. It is debatable whether this is desirable or if, to the contrary, the input dimension should be reduced. We will return to these observations when discussing our pre-

processing measures and address them accordingly.

In addition, it must be taken into consideration that instance distribution across the food groups is not uniform. In fact, the group sizes are heavily skewed: beef products is by far the biggest category featuring 946 instances. Spices and herbs, on the contrary, only consists of 64 items. The initial class distribution is shown in Figure 1. While the 9 classes that take up less than 3% of the data set are grouped, the 3 biggest classes together are almost 30%. The remaining 13 are more or less distributed uniformly.

Preprocessing

As a first step, we removed the columns that were insignificant to the task at hand. We dropped all the columns with additional albeit not nutritional information, namely "ShortDescrip", "Descrip", "CommonName", "ScientificName" and "ID" reducing the number of columns to 38. In the previous section of this report, we briefly touched on the idea of too many features constituting a problem. This is what is commonly referred to as the "curse of dimensionality". I.e., in relation to the numerous dimensions present in the feature set, there is comparatively little data. As a result, classifiers' predictions may deteriorate (Tan et al., 2005).

Tan et al. propose two measures in order to reduce dimensionality: either the use of algebraic approaches such as a principal component analysis (PCA) - which we will discuss later on in this section - or picking a selection of a subset of features. The latter case is based on the prerequisite that the information of some features is contained within others and, therefore, they can be left out (e.g. the price of a product entails the sales tax paid for it). Transferring this knowledge to the nutrition data set at hand, we could already identify one such feature by applying domain knowledge. Evidently, the feature "Energy_kcal" serves merely as a proxy for information contained in the other features (primarily Fat_g, Protein_g and Carb_g). Thus, we also dropped this feature from the data set reducing the number of potential features to 37.

A similar case can be made for all the dietary intake recommendations. Evidently, given that our self-imposed task was to predict the food group based on an instance's nutrients, using the intake recommendations was beyond the scope of our work. Furthermore, as it is the case for "Energy_kcal", the intake recommendations only represent a share of the respective instance's nutritional values, i.e. their entire information is already entailed by the nutrient features they refer to. Thus, we decided to also remove all the USRDA columns leaving a total of 22 features to be used for classification.

Next, we proceeded to examine the remaining data with respect to its consis-

tency. First, we checked for potentially missing values planning to fill them in with the mean values for said feature within the respective group. However, as in the remaining columns all values were present, we were able to skip this step. Similarly, we checked whether there would be any identical instances that may have been duplicated by mistake. Likewise, we did not find any duplicates.

With respect to anomaly detection, we first tried to identify abnormal values by the use of domain knowledge. As the nutritional values for each instance are listed per 100 grams, it is obvious that no feature can be greater than this figure (converting milligrams and micrograms accordingly). We found a series of instances that did indeed contain 100 grams of fat and, in addition, traces of other nutrients. Most of these instances belonged to the fat and oil group. Hence, considering that measurement can not be perfectly accurate especially in regard to nutrients that occur in minute doses (e.g. vitamins), we still consider such an observation to be conceivable and within the range of measuring accuracy. Subsequently, we summed over all values to check whether the sum would exceed 100 grams (excluding "Carb_g" and "Fiber_g", as there is evidently some overlap with other features such as "Sugar_g"). In order to account for potential floating point imprecision, we used the numpy 64bit implementation of float. Again, we found two instances whose sum was slightly above 100.0 grams which we still considered valid items.

Subsequently, we moved on to more technical anomaly detection. Considering each feature as isolated measures within the respective group bears the risk of overlooking the fact that a combination of feature values can also constitute an outlier. Thus, we used Local Outlier Factor (LOF), a density based outlier detection method, to approach this problem. (Breunig et al., 2000). We applied LOF individually to each class receiving one score per instance. The higher the score, the more likely the item is to be an outlier. Then, we transformed the LOF values of each class to normal distribution. This allowed us to determine fringe samples at the margins of the distribution more easily. We eliminated instances whose LOF values in normal distribution were below -0.9 or above 0.9. After applying the outlier detection, the number of instances was reduced to 8178.

Having removed outliers, we then went on to normalize the data. Considering that not only do the features' units differ but also the various value ranges, we normalized the data using min-max-normalization. Thereby, each feature's value range was converted to fit into a common interval of [0, 1]. Note that the order of our pre-processing steps is important here. Normalizing the data first and then identifying and removing outliers would bear the risk of distorted value ranges. Although LOF takes all features into consideration, it is still conceivable that one instance would represent an extreme data point for a single feature. Therefore, normalizing the data first and then removing outliers could mean removing the lower or upper bounds (0 or 1) of the now normalized data.

At the beginning of this section we also talked about the potential use of algebraic approaches to reduce dimensionality. Given the fact that a dimensionality of 22 is still quite big and may cause the aforementioned problems, we decided to also perform a Principal Component Analysis (PCA) as well as a Linear Discriminant Analysis (LDA) in order to reduce dimensionality. PCA can keep the most important features of high-dimensional data, while removing noise and unimportant features and thereby improving data processing speed. The idea of PCA is to map n -dimensional features to k dimensional (with $k < n$), which is a completely new orthogonal feature set. This is called the principal element and is an entirely reconstructed k -dimensional feature, rather than simply removing the remaining $n-k$ dimensional features from the original n dimensions. PCA is intended to create features that are independent from each other. LDA, on the other hand, is used for reducing dimensions before classification. LDA finds linear combinations of features that model the difference between data classes. Unlike PCA, it deals with categorical independent variables. Also, LDA works well with data sets when groups are known a priori.

Initially, we picked the share of components to keep for PCA at 90%. However, after some quick tests, with mediocre results, we resorted to keeping the parameter at 95%. This resulted in an 8-dimensional feature set. In order to achieve comparability between PCA and LDA, we picked the same number of dimensions for LDA where dimensions can be determined manually.

For the next step, we split all of the three data sets into two parts each using a ratio of 70 to 30. The first part would then be used as the training, the second as the test set. Note that in the preceding section, we discussed the data set's structure and also referred to the problem that items are distributed quite unevenly across classes. At this point, we briefly considered the possibility of undersampling in order to establish a more level distribution. However, regarding the great disparity of class sizes, we quickly discarded this idea. Cutting each class's size down to the least frequent one would have been synonymous with disregarding a lot of valid data. Thus, we referred to a different approach instead and performed over-sampling on the training set. The test sets, however, remained unaltered.

We achieved this by applying synthetic minority over-sampling technique (SMOTE) to the three training sets. SMOTE achieves over-sampling by taking each minority class and creating new instances based on the class's k nearest neighbors. The number k of neighbors that are used for sampling is dependent on the amount of over-sampling that has to be performed (Chawla et al., 2002).¹ After over-sampling, each class consisted of 632 instances.

¹In our code, we used the smote feature included in the imbalanced-learn framework. In anaconda, it can be installed via `conda install -c conda-forge imbalanced-learn`

The decision to apply over-sampling also lent further importance to the outlier detection we performed before. Considering that in over-sampling, new data is replicated based on existing instances, the failure to handle outliers bears the risk of said information being reproduced during the procedure - thereby reinforcing outliers' bias (Tan et al., 2005).

Data Mining

As far as the evaluation setup is concerned, we decided to make the following choices. As far as classifiers are concerned, we a priori eliminated models that are typically used for binary classification problems and whose transfer to multi-class approaches are not straight-forward, for example Support Vector Machines and Logistic Regression. The classifiers which we identified as potentially interesting are K-Nearest Neighbours (KNN), Nearest Centroids (NC), Naive Bayes (NB), Decision Trees (DC) and Neural Networks (NN). Then, we determined baseline classifiers for each of the three data representations which a potential model should be able to outperform. Subsequently, we performed a grid search for each classifier candidate using various parameters and each of the three data representations. In order to facilitate discourse throughout the remainder of this report, we will refer to the three data representations as "default", "PCA" and "LDA".

As far as the competitive baseline is concerned, we initially resorted to Scikit-Learn's DummyClassifier for this issue. However, given that we had balanced class distribution and tried to predict into 25 classes, a majority classifier was no longer an option. Similarly, none of the other dummy classifiers contained in Scikit-Learn proved to be a serious lower threshold for our experiments. Thus, we resorted to creating a very simple baseline classifier ourselves. We used a decision tree with a maximum depth of 4 and the gini coefficient as splitting criterion for this. This classifier achieves F1 scores of 0.24, 0.19 and 0.25 on the default data, the PCA data and the LDA data respectively. We consider this to be an acceptable minimum standard that a potential classifier should be able to beat in order to be considered a valid candidate.

For KNN, we varied with two different settings: the weighting method (uniform, i.e. none, and by distance) and the number of neighbours. The latter we tested from 2 to 10. For the default data, $n=3$ with distance weighting performed the best with precision (P), recall (R) and F1 scores of 0.81, 0.80 and 0.80. Note, however, that all scores for $n \in [2, 9]$ were quite close. While PCA performed slightly worse with $n=2$ and distance weighting as the best configuration ($P=0.74$, $R=0.72$, $F1=0.72$), the scores achieved with LDA were quite close to the one determined with the default data, while $k=5$ and distance weighting performed best

here ($P=0.79$, $R=0.78$, $F1=0.78$).

Taking a look at the classification report revealed that for all of the three runs, one class had consistently been predicted the worst. For PCA and LDA, American Indian Food's (a class that consists of a variety of different foods) F1 scores were 0.28 and 0.30. For the default data, F1 was slightly better with 0.42. For most of the classes, however, the results achieved using LDA were on par with the ones when using default. Interestingly enough, for LDA, KNN was significantly better at classifying Restaurant Foods than for default ($F1=0.57$ vs. $F1=0.47$). While both settings showed comparatively good R scores (0.78 for LDA, 0.72 for default), LDA's P was a lot better (0.45 vs 0.34). This means that for this group of foods, with LDA KNN was better at identifying instances that did in fact belong to the class, whereas for the default representation, KNN incorrectly classified more items to be Restaurant Foods that were not. The confusion matrix revealed that many of these false positives belonged to the Fast Food group.

For DC, we tested two different splitting criteria, namely 'gini' and 'entropy' and gradually increased the trees maximum depth starting from 5 to 40. It turned out that on the default data, using entropy as a splitting criterion and a maximum depth of 23 performed the best ($P=0.79$, $R=0.79$ and $F1=0.79$). However, improvement only occurred until a maximum depth of 13. After that, performance was fairly similar for all chosen depths and only slight fluctuations occurred. Moreover, since performance did not deteriorate for greater depths, one can ascertain that no over-fitting took place. For PCA, entropy with a maximum depth of 25 ($P=0.71$, $R=0.69$, $F1=0.69$) and for LDA, entropy with a maximum depth of 26 ($P=0.70$, $R=0.68$, $F1=0.69$) performed best. Again, F1 only improved until a maximum depth of 12 to 13 and results showed only a slight fluctuation for greater maximum depths. However, in total, DC performed a lot better on the default data and, in addition, we could not observe that same improvements for classes with a great variety of foods as it was the case for KNN.

As for NC there are no configuration settings available, we stuck to the standard configuration. It showed by far the worst performance with $P=0.48$, $R=0.38$ and $F1=0.36$ for default, $P=0.42$, $R=0.34$ and $F1=0.32$ for PCA data and $P=0.54$, $R=0.46$ and $F1=0.46$ for LDA. One striking observation is that NC is especially bad for a few groups. Again, American Indian Foods was the worst predicted class with P, R, F1 scores of 0.07, 0.04 and 0.05 on the default data. For LDA, on the other hand, NC performed slightly better for this group. Moreover, it should be mentioned that for PCA, Baby Foods, was the worst, with not a single instance being predicted correctly.

Likewise, for NB no further configurations are available. NB, however, performed significantly better than NC with P, R and F1 of 0.57, 0.46 and 0.46 on the default data (PCA: 0.45, 0.38, 0.37. LDA: 0.47, 0.46, 0.43). Interestingly enough,

for NB, Baby Foods was by far the class with the worst scores for all of the three runs. Note, however, that NB did not predict a single instance of this group correctly using PCA and LDA ($P=R=0$ for both). Most of these items were instead predicted to be Soups and Sweets. Thus, while NB evidently struggles with this class in general, the dimensionality reduction seems to have further exacerbated this problem.

As far as the NN² is concerned, it should be noted that there is the greatest number of possible configurations for each of the tested classifiers. However, given that it is impossible to test all of them, we resorted to a base configuration that we considered sensible for the task at hand. We used two hidden layers with relu activation and used softmax as a final activation function. The number of nodes in the hidden layers was initially set to 16. Then, we proceeded to test the network in 9 iterations, increasing the number of neurons in the hidden layer by 16 in each of them. This way, we ultimately arrived at a maximum number of 160 nodes in the hidden layer. In each iteration, the network was trained for 50 episodes using a batch size of 5 for all three of the reconstructed data. In total, due to lack of a GPU, training all configurations took 5 hours.

For default, the network worked best with 144 nodes ($P=0.84$, $R=0.83$, $F1=0.83$). For 160 nodes, results showed a slight deterioration, possibly the result of over-fitting. While NN for PCA performed the worst for the three data representations, it was still significantly better than prior results achieved using other classifiers ($P=0.77$, $R=0.76$, $F1=0.76$). It is interesting to note that these values were identical for 144 or 160 nodes in the two hidden layers. Hence, since no clear decline was observed with a growing number of nodes, it is conceivable that a greater number of nodes could lead to even better results. As far as LDA is concerned, 144 nodes proved out to be the best configuration resulting in $P=0.80$, $R=0.79$, $F1=0.79$. Again, results of the classifier run on LDA were closer to the ones observed on default.

With regards to the usual classes that had been hard to classify previously, the neural network performed slightly better in some cases: Whilst American Indian Food was still the worst predicted class for all of the three data representations, it was still significantly better than compared to other classifiers. Nevertheless, with 0.35, LDA showed the lowest F1 score for this class. This represents a contrast to other results where prediction results on this class were better using LDA.

²For the neural network, we used Keras's sequential model. In anaconda, keras can be installed via `conda install -c conda-forge keras`

Classifier \ Data	Default	PCA	LDA
Baseline	0.24	0.19	0.25
KNN	0.80	0.72	0.78
NC	0.36	0.32	0.46
NB	0.46	0.37	0.43
DC	0.79	0.69	0.69
NN	0.83	0.76	0.79

Table 1: Classifier F1 scores for the best configurations using the default data, PCA and LDA

The summarized results of the classifier configurations that we have found to perform best are depicted in Table 1. Comparing the final results of our tests, it can be ascertained that all of the tested classifiers outperformed the competitive baseline on default, PCA and LDA. However, NB and NC still exhibited the worst performance among the classifiers. KNN, DC and NN, on the other hand, were closer in their results on all three data representations.

As far as dimensionality reduction is concerned, it can be determined that PCA and LDA did not achieve the desired improvements. Especially for PCA, results were almost always worse than for default. LDA’s performance, however, was consistently closer to default. Moreover, for some classifiers, making predictions on LDA showed more promising results for classes that feature a great variety of different foods, as it is the case with American Indian Foods. Given that we initially identified this as a possible difficulty for the project, we still think that there is some leeway for improvements that can be achieved - possibly by other forms of dimensionality reduction.

In our initial project proposal, we voiced the concern that it would be especially difficult to differentiate between different types of meat. This problem however, did not occur. More often than not, meat groups were predicted quite reliably except for NC. Moreover, while we did anticipate potential problems with foods that show a great deviation from their respective group, we did not expect the issue to be this severe. Especially groups that serve as an umbrella for a greater variety of foods are often times predicted quite poorly (i.e. American Indian Foods or Baby Foods). Again, this is most likely caused by the fact that these groups comprise a great variety of foods. In addition, back then we did not foresee that a class number of 25 would pose a problem.

Ultimately, we ascertain that we succeeded in our initial goal to find a sufficiently strong classifier that is able to predict a food’s group based on its nutritional

values in the NNDB. Nevertheless, given that the best achieved F1 score is 0.83, we think that there is still room for improvements. Possibly other dimensionality reduction methods, that did not find application within this work, may be able to further improve the results.

Bibliography

Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. Lof: Identifying density-based local outliers. *SIGMOD Rec.*, 29(2):93–104, May 2000. ISSN 0163-5808. doi: 10.1145/335191.335388.

Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357, 2002.

Craig Kelly. Usda national nutrient db. <https://data.world/craigkelly/usda-national-nutrient-db>, Dec 2016.

Pang-Ning Tan, Michael Steinbach, and Vipin Kumar. *Introduction to Data Mining, (First Edition)*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2005. ISBN 0321321367.